Natural Language Processing (Fall-2026)

Assignment-1

Instructor: Dr. Hajra Waheed

Teacher Assistant

- **Sidra Saleem:** l258009@lhr.nu.edu.pk
- For **assignment & Viva-related queries**, please contact the TA directly via email.

Submission Guidelines:

- Submit your assignment on Google Classroom in the format "20XX.ipynb".
- **The deadline is September 12, 2026, at 11:59 PM. No extensions will be granted.**

Declarations:

- Late submissions will incur penalties: 25% deduction on the first day, 50% on the second day, and zero marks thereafter.
- **Plagiarism will result in zero marks for the assignment.**
- This is an individual assignment; collaboration or group work is strictly prohibited.
- Please ensure that you submit your own original work.

VIVA Policy:

- A VIVA (oral examination) will be conducted to assess your understanding of the assignment.
- The VIVA will be scheduled separately, and you will be notified of the date and time.
- Failure to attend the VIVA will result in zero marks for the assignment.

Academic Integrity:

- Plagiarism, collusion, and academic dishonesty will not be tolerated.
- Any instances of academic misconduct will be reported to the authorities and may result in severe penalties.

Objective

The objective of this assignment is to develop a strong practical and conceptual foundation in Natural Language Processing (NLP). Students will explore the complete NLP pipeline, starting from raw text acquisition and preprocessing to statistical and neural language modeling, and subword segmentation techniques.

Through hands-on implementation, students will:

- Apply regular expressions for pattern matching and information extraction
- Perform text preprocessing, normalization, tokenization, stemming, lemmatization, POS tagging, and named entity recognition
- Build and evaluate statistical language models (unigram and bigram) with Laplace smoothing and perplexity analysis
- Implement Byte Pair Encoding (BPE) for subword modeling, including Urdu Unicode text processing
- Tokenize Urdu text and generate n-gram models (unigrams, bigrams, trigrams) for a non-Latin script language

By the end of this assignment, students will gain the ability to design, implement, evaluate, and critically analyze NLP models, bridging classical techniques with modern subword and language modeling approaches used in real-world NLP systems and research.

Part 1: Regular Expressions and Preprocessing (35%)

This part introduces the complete **text preprocessing pipeline** used in NLP: acquiring text, cleaning and normalization, regex-based pattern extraction, tokenization, stopword handling, stemming/lemmatization, and basic linguistic annotation (POS + NER). Students will also perform **small error analysis** to develop evaluation habits.

Tools & Required Libraries

- Nltk
- beautifulsoup4
- punkt / punkt_tab
- stopwords
- averaged_perceptron_tagger
- maxent_ne_chunker
- words
- wordnet
- omw-1.4

Dataset / Text Source (Required)

Use one public web page as your dataset (English, non-poetry), such as:

- Stanford NLP Group people page (example): https://nlp.stanford.edu/people/
- If it does not work, use any other informational webpage **URL** (university department page, organization page, research lab page).

Constraints

- Use a webpage containing real text and ideally contact info and other textual information as well which can satisfy tasks.
- You must include the final URL used in your report.

Task:P1T1. HTML to Plain Text Utility

Write a function: def fetch_clean_text(url: str) -> str:

It should:

- 1) download HTML using urllib.request
- 2) extract text using BeautifulSoup(html).get_text()
- 3) return plain text

Deliverable:

print the first 400 characters of cleaned text and before cleaning

Task:P1T2. Tokenization + Regex Extraction (Emails & Phones)

Using text from P1T1:

- 1. Email + Phone extraction
- 2. Tokenization: Tokenize using `nltk.word_tokenize()` and `nltk.sent_tokenize()`.

Important: Do NOT tokenize before extracting emails (emails break during tokenization). Extract using regex on the **raw cleaned text**.

Helpful Resources:

The following resources may help you understand regular expressions and tokenization concepts. These are for guidance only — you must write your own code.

- Python `re` module documentation: https://docs.python.org/3/library/re.html
- Regex101 (interactive regex tester): https://regex101.com/ — use this to test and debug your patterns
- NLTK Tokenization documentation: https://www.nltk.org/api/nltk.tokenize.html
- Jurafsky & Martin, Chapter 2 (Regular Expressions, Text Normalization): https://web.stanford.edu/~jurafsky/slp3/2.pdf
- RFC 5322 (email format specification) — useful for understanding valid email patterns
- *Hint:* Consider edge cases in email patterns such as subdomains (e.g., user@cs.uni.edu) and phone formats with country codes, parentheses, or hyphens.

Deliverable:

- Total sentences
- Total tokens
- List of unique emails
- List of unique phone numbers
- Show your regex patterns

Task:P1T3. Text Normalization Pipeline

Using text from P1T2:

Normalization must include:

- lowercasing
- remove extra whitespace
- optional: replace digits with <NUM>

- optional: replace URLs with <URL>
- optional: remove punctuation (must justify choice)

Deliverable:

- show before/after snippet (first 250 chars)
- mention which rules you applied and why

Task:P1T4. Stopwords and Corpus Statistics

Using text from P1T3:

Given a text corpus, perform the following steps:

- 1. Compute the top-15 most frequent tokens before stopword removal.
- 2. Remove English stopwords using NLTK.
- 3. Compute the top-15 most frequent tokens after stopword removal.

Metrics to Report

- Total token count (before and after stopword removal)
- Vocabulary size (number of unique tokens, before and after)
- Type–Token Ratio (TTR) for both cases

TTR = (Number of unique tokens) / (Total number of tokens)

Deliverables

- Two top-20 frequency lists (before vs after stopword removal)
- A statistics summary table

Task:P1T5. POS Tagging + POS Distribution

Using text from P1T3:

Perform POS tagging on normalized tokens:

- `nltk.pos_tag(tokens)`

Deliverable:

- Top 8 POS tags by frequency
- Extract 12 nouns and 12 adjectives (examples)
- POS distribution table
- Example noun/adjective lists

Task:P1T6. Lemmatization vs Stemming

Using text from P1T3:

Apply the following normalization techniques to a set of 150 tokens:

- **Porter Stemmer**
- **Snowball Stemmer**
- **WordNet Lemmatizer**

Hint: You can use the following NLTK classes for stemming and lemmatization:

- `from nltk.stem import PorterStemmer` — for Porter stemming
- `from nltk.stem import SnowballStemmer` — initialize with SnowballStemmer("english")
- `from nltk.stem import WordNetLemmatizer` — for lemmatization; consider passing the POS tag to `lemmatize()` for more accurate results

Deliverables

1. A **comparison table** with the following columns:
    - Token
    - Porter Stem
    - Snowball Stem
    - Lemma

Token	Porter	Snowball	Lemma

2. A brief **discussion** (3–5 sentences): In which cases does the Porter Stemmer produce a different result from the Snowball Stemmer? When does lemmatization preserve meaning better than stemming?

Task:P1T9. Named Entity Recognition (NER)

Perform NER using NLTK chunker:

- `nltk.ne_chunk(pos_tags)`

Extract and report unique named entities under:

- PERSON
- ORGANIZATION
- GPE/LOCATION
- DATE

Deliverable:

- Entity lists + counts
- Brief discussion: which entity types were most/least accurately detected and why?

Part 2 (Language Models and Smoothing) 30%

This question requires you to train some language models on a training corpus and to test them on two smaller corpora. Starter code is also provided with this assignment. Each sentence must be surrounded by a start of sentence and end of sentence marker (<s> ... </s>). Preprocess to add these sentence markers. These markers will allow your models to generate sentences that have realistic beginnings and endings.

Dataset

Brown Corpus (NLTK) — use only the **"news"** and **"editorial"** categories

Inaugural Address Corpus (NLTK) / ABC News Corpus (NLTK) **for out of Domain Test**

Implement the following models:

- 1. UnigramModel: an unsmoothed unigram model
- 2. SmoothedUnigramModel: a unigram model smoothed using Laplace (add-one) smoothing
- 3. BigramModel: an unsmoothed bigram model
- 4. SmoothedBigramModel: a bigram model smoothed using Laplace (add-one) smoothing

For each of the four language models, you need to implement the following methods:

- **generateSentence(self)**: returns a sentence sent that is generated by the language model. sent is a list of the form [<s> w1, ..., wn </s>]. You can assume that <s> starts each sentence (with probability 1). The following words (<s> w1, ..., wn </s>) are generated according to your language model's distribution. The number of words (n) is not fixed. Instead, you stop generating a sentence as soon as you generate the end of sentence symbol </s>.
- **getSentenceProbability(self, sen)**: returns the probability of the sentence sen (which is again a list of the form [<s> w1, ... , wn </s>]) according to the model. Please use the provided generateSentencesToFile method and your unigram and bigram language models to generate 25 sentences (saved as unigram_output.txt, smoothunigram_output.txt, bigram_output.txt and smoothbigram_output.txt).

Perplexity

You need to compute the perplexity (normalized inverse log probability) of the two test corpora according to all of your models. Evaluate the models on the test corpora. Do you see a difference between the two test domains?

Questions

- 1. When generating sentences with the unigram model, what controls the length of the generated sentences? How does this differ from the sentences produced by the bigram models?

- 2. Consider the probability of the generated sentences according to your models. Do your models assign drastically different probabilities to the different sets of sentences? Why do you think that is?
- 3. Generate additional sentences using your unsmoothed bigram and Laplace-smoothed bigram models. In your opinion, which model produces better / more realistic sentences? Justify your answer with specific examples.
- 4. For each of the four models, which test corpus has a higher perplexity? Why? Make sure to include the perplexity values in the answer.
- 5. How does Laplace smoothing affect the probability distribution compared to the unsmoothed models? Discuss the impact on both common and rare n-grams with examples from your output.
- 6. What happens when your unsmoothed bigram model encounters a bigram at test time that was never seen during training? How does the Laplace-smoothed bigram model handle this differently? Illustrate with a concrete example from your test corpora.

Part 3: Byte Pair Encoding & Urdu Text Processing (35%)

The objective of this part is to provide hands-on experience with Unicode text processing using the Byte Pair Encoding (BPE) algorithm and n-gram generation for non-Latin script languages. BPE is a data-driven subword tokenization technique widely used in English NLP, where it helps reduce vocabulary size, handle rare words, and improve generalization by decomposing words into frequent subword units.

In this assignment, BPE is applied to Urdu Unicode text to understand subword segmentation in the context of a morphologically rich and low-resource language. Students will learn how BPE merges character sequences to form meaningful subwords, while correctly handling Unicode normalization, token boundaries, and script-specific characteristics of Urdu.

This part highlights how BPE enables better vocabulary coverage, reduced out-of-vocabulary (OOV) rates, and more robust representations, both in English and Urdu, and prepares students for modern embedding-based NLP models.

Helping Material

For background on the BPE algorithm, refer to Chapter 2 of the course textbook: Speech and Language Processing (Jurafsky & Martin) https://web.stanford.edu/~jurafsky/slp3/

Use a good Unicode text editor (such as **BabelPad**) to view Urdu files.

Task:P3T1 — Byte Pair Encoding (BPE) for Urdu Word Segmentation

In this task, you are provided with 50 unsegmented Urdu sentences along with a dictionary (wordlist). Your objective is to implement the Byte Pair Encoding (BPE) algorithm to segment the unsegmented Urdu text into meaningful subword units and words.

You must apply BPE-based segmentation to each sentence and generate a segmented version of the input text. You may preprocess the dictionary to simplify or optimize the lookup process; however, any preprocessing steps or assumptions must be clearly explained in no more than 100 words in your report.

Input Files

- `wordlist.txt` — Urdu dictionary for lookup
- `50_nospaces.txt` — 50 Urdu sentences without spaces

Output

- `segmented_output.txt`
  - Contains the segmented version of each input sentence
  - One segmented sentence per line
  - UTF-8 encoded

Additional Deliverable:

- Report the top-10 most frequent BPE merge operations performed during segmentation and briefly explain their linguistic significance.

Task: P3T2 — Average Characters per Word in Urdu Text

In this task, you are required to calculate the average number of characters per word in a running Urdu text. You must collect a large Urdu corpus containing at least 12,000 tokens from the internet, such as news articles or social media posts (poetry is not allowed).

After tokenizing the text, compute the number of **Unicode characters per token** and report the **average characters per token**.

Input

- A self-collected Urdu text corpus (≥ 12,000 tokens)

Output

- Total number of tokens
- Average number of characters per token
- Source link(s) of the corpus used

All results must be clearly reported in the written report.

Task: P3T3 — Urdu Text Tokenization and N-Gram Generation

In this task, you will tokenize a given Urdu text and generate **unigram**, **bigram**, and **trigram** models from the tokenized output. This task tests your ability to handle tokenization and n-gram extraction for a right-to-left, space-delimited (but morphologically complex) language.

Input

Use the same Urdu corpus you collected in Task P3T2 (≥ 12,000 tokens). If your corpus from P3T2 is not suitable for this task, you may collect additional Urdu text from news websites.

Steps

- 1. **Tokenization:** Tokenize the Urdu text into individual words/tokens. You may use whitespace-based tokenization or a more advanced approach (e.g., regex-based tokenization that handles Urdu punctuation marks such as ۔ and ، correctly). Clearly state and justify your tokenization strategy.
- 2. **Preprocessing:** Remove any non-Urdu characters (English text, numbers, special symbols) from the tokenized output. Normalize Unicode characters where necessary (e.g., handling different forms of ی and ک).
- 3. **Unigram Generation:** Compute unigram (single token) frequencies from the tokenized corpus. Report the top-20 most frequent unigrams.
- 4. **Bigram Generation:** Compute bigram (two consecutive tokens) frequencies. Report the top-20 most frequent bigrams.
- 5. **Trigram Generation:** Compute trigram (three consecutive tokens) frequencies. Report the top-20 most frequent trigrams.

Deliverables

- Total token count after preprocessing
- Total unique tokens (vocabulary size)
- Top-20 unigrams with frequencies
- Top-20 bigrams with frequencies
- Top-20 trigrams with frequencies
- A brief **analysis** (5–8 sentences) discussing:
    - What do the most frequent unigrams tell you about Urdu text structure? Are function words (e.g., □ □□) dominant, similar to English stopwords?
    - Do the frequent bigrams/trigrams correspond to meaningful phrases or common grammatical patterns in Urdu?
    - Compare the n-gram distributions: how quickly do frequencies drop off from unigrams to bigrams to trigrams? What does this imply about data sparsity in Urdu language modeling?

Hint: You can use Python's `collections.Counter` for frequency counting and NLTK's `nltk.ngrams()` or `nltk.bigrams()` / `nltk.trigrams()` utility functions. However, you must handle Urdu-specific preprocessing (Unicode normalization, punctuation) yourself.

Deliverables

1. 1 Notebook

- A single notebook containing code implementations for all parts, organized sequentially by the specified TASK IDs.
- All results must be visible and reproducible, with every cell already executed.
- The notebook must not require re-running to verify outputs.

2. 1 Detailed Report

- A well-structured written report covering each task separately.
- Include screenshots of results and outputs for every task.
- Ensure proper academic formatting, clear headings, and consistent labeling.

3. Exported Files

- All required exported artifacts (e.g., generated data files, plots, models, or result files) referenced in the notebook and report.

Screenshots in the report should correspond to key results and tables shown in the notebook.

Important Notes

- Incomplete execution, missing TASK IDs, or improperly formatted reports may result in loss of marks.
- Ensure consistency between notebook outputs, screenshots, and exported files.